All use of pydantic classes + validation

And all use of annotation

At second last separate example how pydantic class with pathparameter query parameter or headers and others and

At last also complete and wide example of perfect pydantic and annoatted use

PART 1 — All Common Uses of `Annotated` in FastAPI

General syntax:

```
Annotated[TYPE, FASTAPI_PARAMETER()]
```

Meaning:

```
TYPE → expected data type
FASTAPI_PARAMETER → where the data comes from
```

---

1 Query Parameters — `Query()`

Used when data comes from **URL query string**.

Example request:

```
GET /products?limit=10&category=electronics
```

Example:

```
from fastapi import FastAPI, Query
```

```python
from typing import Annotated

app = FastAPI()

@app.get("/products")
async def get_products(
    limit: Annotated[int, Query(gt=0, lt=100)],
    category: Annotated[str | None, Query()] = None
):
    return {
        "limit": limit,
        "category": category
    }
```

Validation options

Parameter	Meaning
gt	greater than
ge	greater or equal
lt	less than
le	less or equal
min_length	minimum string length
max_length	maximum string length
regex	pattern validation

Example:

```python
username: Annotated[str, Query(min_length=3, max_length=20)]
```

---

2 Path Parameters — `Path()`

Used for **URL path values**.

Example request:

```
GET /products/5
```

Example:

```python
from fastapi import Path
```

```
@app.get("/products/{product_id}")
async def get_product(
    product_id: Annotated[int, Path(gt=0)]
):
    return {"product_id": product_id}
```

Validation:

```
product_id: Annotated[int, Path(gt=0, le=1000)]
```

Meaning:

```
product_id must be between 1 and 1000
```

---

3 Request Body — `Body()`

Used when data comes from **JSON body**.

Example request:

```
POST /products
```

Body:

```
{
 "name": "Laptop",
 "price": 1200
}
```

Example:

```
from fastapi import Body
from pydantic import BaseModel

class Product(BaseModel):
    name: str
    price: float

@app.post("/products")
async def create_product(
```

```
    product: Annotated[Product, Body()]
):
    return product
```

Why use Body?

To add **extra validation or metadata**.

Example:

```
price: Annotated[float, Body(gt=0)]
```

---

4 Headers — `Header()`

Used to read **HTTP headers**.

Example request:

```
Authorization: Bearer abc123
```

Example:

```
from fastapi import Header

@app.get("/user")
async def get_user(
    authorization: Annotated[str, Header()]
):
    return {"token": authorization}
```

Optional header:

```
user_agent: Annotated[str | None, Header()] = None
```

`we can test in the curl we mentioned in the header chapter`

---

5 Cookies — `Cookie()`

Used to read **browser cookies**.

Example request:

```
Cookie: session_id=abc123
```

Example:

```
from fastapi import Cookie

@app.get("/session")
async def get_session(
    session_id: Annotated[str | None, Cookie()] = None
):
    return {"session_id": session_id}
```

Used for:

- login sessions
- tracking users

---

6 Dependencies — `Depends()`

Used for **dependency injection**.

Example:

```
from fastapi import Depends

async def get_token():
    return "abc123"

@app.get("/secure")
async def secure_route(
    token: Annotated[str, Depends(get_token)]
):
    return {"token": token}
```

Used for:

- authentication
- database connections
- reusable logic

---

7 Form Data — `Form()`

Used when receiving **HTML form data**.

Example request:

```
POST /login
Content-Type: application/x-www-form-urlencoded
```

Example:

```
from fastapi import Form

@app.post("/login")
async def login(
    username: Annotated[str, Form()],
    password: Annotated[str, Form()]
):
    return {"username": username}
```

Used for:

- login forms
- web forms

---

8 File Upload — `File()`

Used for **uploading files**.

Example:

```
from fastapi import File, UploadFile

@app.post("/upload")
async def upload_file(
    file: Annotated[UploadFile, File()]
):
    return {"filename": file.filename}
```

Used for:

- images
- documents
- videos

SUMMARY — All Annotated Types

Type	Use
Query()	query parameters
Path()	URL path values
Body()	JSON body
Header()	HTTP headers
Cookie()	cookies
Depends()	dependency injection
Form()	form data
File()	file upload

PART 2 — Pydantic Models + Validation

Pydantic is used for:

```
validation
data parsing
structured models
```

Example 1 — Base User Model

```
from pydantic import BaseModel, Field, EmailStr

class UserBase(BaseModel):
    username: str = Field(min_length=3, max_length=20)
    email: EmailStr
    age: int = Field(gt=0, lt=120)
```

Validation here

Field	Validation
username	3–20 characters
email	valid email
age	between 1 and 119

Example request:

```
{
 "username": "ali",
 "email": "ali@test.com",
 "age": 25
}
```

Example 2 — Inherited Model (UserCreate)

Real APIs often use **inheritance**.

```
class UserCreate(UserBase):
    password: str = Field(min_length=8)
```

Meaning:

```
UserCreate inherits all fields from UserBase
```

Now fields:

```
username
email
age
password
```

Example request:

```
{
 "username": "ali123",
 "email": "ali@test.com",
 "age": 25,
 "password": "mypassword123"
}
```

Example 3 — Response Model (UserResponse)

We **hide password** in response.

```
class UserResponse(UserBase):
    id: int
```

Now response contains:

```
id
username
email
age
```

Real FastAPI Example

```
from fastapi import FastAPI
from typing import Annotated

app = FastAPI()

@app.post("/users")
async def create_user(
    user: Annotated[UserCreate, Body()]
):
    return {
        "message": "User created",
        "user": user
    }
```

More Pydantic Validations

Numeric

```
price: float = Field(gt=0)
```

Meaning:

```
price must be greater than 0
```

String Length

```
name: str = Field(min_length=2, max_length=50)
```

Regex

```
phone: str = Field(pattern="^[0-9]{10}$")
```

Valid:

```
1234567890
```

Invalid:

```
abc123
```

Default Values

```
is_active: bool = True
```

Lists

```
tags: list[str] = []
```

Example:

```
{
 "tags": ["electronics", "sale"]
}
```

Example — Product Model

```
class Product(BaseModel):
    name: str = Field(min_length=3, max_length=50)
    price: float = Field(gt=0)
    quantity: int = Field(ge=0)
```

```
    tags: list[str] = []
```

Example request:

```
{
 "name": "Laptop",
 "price": 1200,
 "quantity": 10,
 "tags": ["electronics", "computer"]
}
```

Final Example — Complete API

```
from fastapi import FastAPI, Body
from typing import Annotated

app = FastAPI()

@app.post("/products")
async def create_product(
    product: Annotated[Product, Body()]
):
    return product
```

Final Summary

Annotated tells FastAPI:

```
Where data comes from
```

Examples:

```
Annotated[int, Query()]
Annotated[str, Header()]
Annotated[User, Body()]
Annotated[str, Depends()]
```

Pydantic Models provide:

```
data validation
data structure
automatic parsing
```

With validation like:

```
min_length
max_length
gt
lt
regex
email
lists
default values
```

USE OF ANNOTATED + PYDANTIC CLASSES(different example like pass of path parameter , query parameter , header etc…..with annotation)

1 First Understand the Roles

Pydantic Model

Used for:

- request body validation
- structured data
- schemas
- response models

Example:

```
from pydantic import BaseModel

class Product(BaseModel):
    name: str
    price: float
```

Annotated

Used to attach **FastAPI metadata** such as:

- Query()
- Header()
- Path()
- Body()
- Depends()

Structure:

```
Annotated[type, FastAPI metadata]
```

---

2 Real-World Example (Body + Pydantic + Annotated)

```
from fastapi import FastAPI, Body
from typing import Annotated
from pydantic import BaseModel

app = FastAPI()

class Product(BaseModel):
    name: str
    price: float

@app.post("/products")
async def create_product(
    product: Annotated[Product, Body()]
):
    return product
```

Why use Annotated here?

```
Annotated[Product, Body()]
```

It tells FastAPI:

```
This Product model comes from the request body
```

Even though FastAPI can infer it automatically, **this is the modern recommended style**.

---

3 Real-World Example (Query + Pydantic Model)

Large APIs often group query parameters into a model.

```
from fastapi import Query
from typing import Annotated
from pydantic import BaseModel

class ProductFilter(BaseModel):
    limit: int = 10
    offset: int = 0
```

Use it:

```
@app.get("/products")
async def get_products(
    filters: Annotated[ProductFilter, Query()]
):
    return filters
```

Now query parameters become structured.

Request:

```
/products?limit=5&offset=10
```

Response:

```
{
 "limit": 5,
 "offset": 10
}
```

---

4 Real-World Example (Headers + Pydantic)

You already saw this pattern.

```python
class ProductHeaders(BaseModel):
    authorization: str
    accept_language: str | None = None
```

Dependency:

```python
from fastapi import Header, Depends

async def get_headers(
    authorization: Annotated[str, Header()],
    accept_language: Annotated[str | None, Header()] = None
):
    return ProductHeaders(
        authorization=authorization,
        accept_language=accept_language
    )
```

Endpoint:

```python
@app.get("/products")
async def get_products(
    headers: Annotated[ProductHeaders, Depends(get_headers)]
):
    return headers
```

Here:

```
Annotated[ProductHeaders, Depends()]
```

tells FastAPI:

```
This object comes from dependency injection
```

---

5 Real-World Example (Authentication)

Example with **User model**.

```python
class User(BaseModel):
    id: int
    name: str
```

Dependency:

```
async def get_current_user():
    return User(id=1, name="Ali")
```

Endpoint:

```
@app.get("/profile")
async def profile(
    user: Annotated[User, Depends(get_current_user)]
):
    return user
```

Here:

```
Annotated[User, Depends()]
```

means:

```
inject authenticated user
```

This pattern is used in **almost every production API**.

---

6 Real-World Example (Body Validation)

```
class Login(BaseModel):
    email: str
    password: str


Endpoint:

@app.post("/login")
async def login(
    data: Annotated[Login, Body()]
):
    return data
```

```
Client request:
POST /login
Content-Type: application/json


{
 "email": "user@test.com",
 "password": "123456"
}
```

FastAPI automatically:

- parses JSON
- validates data
- creates Login object

---

7 Where `Annotated` Is Common in Real APIs

Use Case	Example
Query parameters	`Annotated[int, Query()]`
Headers	`Annotated[str, Header()]`
Path parameters	`Annotated[int, Path()]`
Request body	`Annotated[Model, Body()]`
Dependencies	`Annotated[User, Depends()]`

---

8 Why FastAPI Recommends `Annotated`

FastAPI documentation prefers this style because:

1 Cleaner type hints

Type stays separate.

```
Product
User
int
str
```

---

2 Better static typing

Works better with:

- VSCode
- mypy
- PyCharm

---

3 Multiple metadata support

Example:

`Annotated[str, Header(), SomeOtherMetadata()]`

---

4 Modern Python typing standard

`Annotated` is the **future-proof approach**.

---

9 Important Thing To Know

If you use a **Pydantic model as request body**, you usually **don't need Body()**.

FastAPI automatically assumes body.

Example:

```
@app.post("/products")
async def create_product(product: Product):
    return product
```

But modern style prefers:

```
product: Annotated[Product, Body()]
```

🔟 Simple Rule

Think like this:

```
Pydantic Model → structure + validation
Annotated → tells FastAPI where data comes from
```

Example:

```
Annotated[Product, Body()]
Annotated[str, Header()]
Annotated[int, Query()]
Annotated[User, Depends()]
```

✅ Final Answer

Yes — **in real-world FastAPI projects we frequently use `Annotated` with Pydantic models**.

Especially for:

- request bodies
- query models
- authentication
- dependency injection
- headers
- path parameters

It is now considered the **recommended modern FastAPI style**.

Complete example :

- **ALL validation is inside Pydantic models**
- `Annotated` is used **only to indicate Body()**
- No extra validation inside `Annotated`
- Clean and **real production style**

This is actually **the recommended way** in many projects.

---

1 Imports

```
from fastapi import FastAPI, Body
from typing import Annotated, List, Optional
from pydantic import BaseModel, Field, EmailStr
from datetime import date
```

---

2 Address Model

All validation is done **inside Pydantic**.

```
class Address(BaseModel):

    street: str = Field(
        min_length=5,
        max_length=100,
        description="Street address",
        examples=["123 Main Street"]
    )

    city: str = Field(
        min_length=2,
        max_length=50,
        description="City name",
        examples=["New York"]
    )

    country: str = Field(
        min_length=2,
        max_length=50,
        description="Country name",
        examples=["USA"]
    )

    postal_code: str = Field(
        pattern=r"^[0-9]{5}$",
        description="5 digit postal code",
```

```
        examples=["10001"]
    )
```

---

3 Phone Model

```
class Phone(BaseModel):

    country_code: str = Field(
        pattern=r"^\+\d{1,3}$",
        description="Country calling code",
        examples=["+1"]
    )

    number: str = Field(
        pattern=r"^\d{10}$",
        description="10 digit phone number",
        examples=["1234567890"]
    )
```

---

4 Base User Model

```
class UserBase(BaseModel):

    username: str = Field(
        min_length=3,
        max_length=20,
        description="Unique username",
        examples=["john_doe"]
    )

    email: EmailStr = Field(
        description="User email address",
        examples=["john@example.com"]
    )

    age: int = Field(
        gt=0,
        lt=120,
```

```
        description="User age",
        examples=[25]
    )

    bio: Optional[str] = Field(
        default=None,
        max_length=200,
        description="User biography",
        examples=["Backend developer"]
    )

    is_active: bool = Field(
        default=True,
        description="Account active status"
    )

    date_of_birth: Optional[date] = Field(
        description="Date of birth",
        examples=["1998-04-21"]
    )

    address: Address

    phones: List[Phone]

    tags: List[str] = Field(
        default=[],
        description="User interests",
        examples=[["python", "fastapi"]]
    )
```

---

5 Create Model (Inheritance)

```
class UserCreate(UserBase):

    password: str = Field(
        min_length=8,
        max_length=50,
        description="User password",
        examples=["StrongPass123"]
    )
```

---

6 Response Model

Password should **not be returned**.

```
class UserOut(UserBase):

    id: int = Field(
        description="Unique user ID",
        examples=[1]
    )
```

---

7 FastAPI App

```
app = FastAPI()
```

---

8 Endpoint Using `Annotated` (Clean Version)

Notice:

- `Annotated` only uses `Body()`
- No validation here
- Everything handled in **Pydantic**

```
@app.post("/users/", response_model=UserOut)
async def create_user(
    user: Annotated[UserCreate, Body()]
):
    return {
        "id": 1,
        **user.model_dump(exclude={"password"})
    }
```

---

9 Example Request

```
POST /users
```

```
{
  "username": "john_doe",
  "email": "john@example.com",
  "age": 25,
  "bio": "Backend developer",
  "is_active": true,
  "date_of_birth": "1998-04-21",
  "address": {
    "street": "123 Main Street",
    "city": "New York",
    "country": "USA",
    "postal_code": "10001"
  },
  "phones": [
    {
      "country_code": "+1",
      "number": "1234567890"
    }
  ],
  "tags": ["python", "fastapi"],
  "password": "StrongPass123"
}
```

---

10 Example Validation Error

```
Bad request:
{
 "username": "a",
 "email": "wrong",
 "age": -5
}
Response:
{
 "detail": [
  {
   "loc": ["body", "username"],
   "msg": "String should have at least 3 characters"
  },
  {
   "loc": ["body", "email"],
   "msg": "value is not a valid email address"
  },
```

```
  {
   "loc": ["body", "age"],
   "msg": "Input should be greater than 0"
  }
 ]
}
```

---

11 Clean Architecture Flow

```
Client Request
      ↓
FastAPI Endpoint
      ↓
Pydantic Validation
      ↓
Service / Business Logic
      ↓
Database
```

---

12 Best Practice Rule

✓ Pydantic handles validation

Examples:

```
min_length
max_length
gt
lt
regex
EmailStr
nested models
lists
default values
```

✓ Annotated only tells FastAPI where data comes from

Example:

```
Annotated[UserCreate, Body()]
```

✅ Final Recommended Pattern

This is the **clean production version**:

```
@app.post("/users/", response_model=UserOut)
async def create_user(
    user: Annotated[UserCreate, Body()]
):
    return service.create_user(user)
```

All validation → **Pydantic**
Annotated → **simple Body()**

If you want, I can also show you the **complete professional FastAPI schema pattern used in large projects**:

```
UserBase
UserCreate
UserUpdate
UserInDB
UserResponse
```

This pattern is **used in real production backend systems**.